

Structure-Aware Chunking for Code RAG: Empirical Evidence from a Legacy Python Codebase

Authors: [Author Names]

Affiliation: [Institution]

Contact: [Email]

Abstract

Retrieval-Augmented Generation (RAG) has emerged as a critical architecture for grounding language model outputs in external code corpora. However, the chunking strategy — the process of dividing source code into retrievable units — remains underexplored as a design variable. We present a systematic empirical comparison of Structure-Aware Chunking (SAC), which segments code along Abstract Syntax Tree (AST) boundaries with hierarchical metadata injection, against Fixed-Size Chunking (FSC), which applies uniform token-window splitting. Experiments are conducted on psf/requests v2.7.0 across four chunk sizes (128, 256, 500, 1500 tokens), two embedding models (MiniLM-384 and Jina-768), and three evaluation phases: retrieval quality (Hit Rate, MRR, nDCG), structural integrity (Syntax Error Rate, Orphan Code Ratio), and generation quality (Answer Relevance, Faithfulness). Our results show that SAC outperforms FSC by 2.0–3.2× in Hit Rate@5 and reduces Syntax Error Rate by up to 63.19 percentage points. A mechanism decomposition analysis reveals that SAC's retrieval advantage operates through two independent layers: structural metadata preservation (active at all chunk sizes) and AST boundary-level segmentation (active at chunk_size ≥ 500). We further find that generation quality does not directly reflect retrieval precision differences, revealing a fundamental trade-off between structural precision for retrieval and contextual density for generation.

Keywords: Retrieval-Augmented Generation, Code Chunking, Abstract Syntax Tree, Information Retrieval, Structural Integrity

1. Introduction

Large Language Models (LLMs) have demonstrated impressive capabilities in code-related tasks, but their reliance on static parametric knowledge limits their ability to reason over specific, evolving codebases. Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), addresses this limitation by grounding generation in dynamically retrieved context from an external corpus. In code domains, RAG pipelines must navigate a fundamental challenge: source code is structured text with syntactic and semantic dependencies that span multiple lines and logical boundaries.

A critical yet underexplored component of code RAG pipelines is **chunking** — the process of dividing source code into retrievable units. The choice of chunking strategy directly determines what context the retriever can surface, and consequently what information is available to the generator. Despite its importance, most deployed RAG systems rely on generic fixed-size chunking strategies that treat code as undifferentiated text, fragmenting function bodies, class definitions, and logical units at arbitrary token boundaries.

Recent work has begun to address this gap. The cAST system (Xie et al., 2025) demonstrates that AST-based chunking improves retrieval recall and generation quality in repository-level code completion tasks, reporting gains of 4.3 points on Recall@5 and 2.67 points on Pass@1 on SWE-bench. However, the mechanisms underlying this improvement — and in particular whether the gain is attributable to boundary-level segmentation, structural metadata, or their combination — remain unclear.

This paper presents a systematic empirical investigation of Structure-Aware Chunking (SAC) versus Fixed-Size Chunking (FSC) applied to a legacy Python codebase. Our contributions are as follows:

1. A three-phase evaluation framework covering retrieval quality, structural integrity, and generation quality applied to a single real-world codebase under controlled conditions.
 2. A **mechanism decomposition analysis** that separates SAC's retrieval advantage into two independent layers: metadata preservation and boundary-level segmentation.
 3. Empirical evidence that structural metadata injection alone — without AST boundary alignment — provides substantial retrieval gains (2–3× Hit Rate@5 improvement).
 4. Identification of a fundamental trade-off between chunking for retrieval precision and chunking for generation quality, with implications for RAG pipeline design.
-

2. Related Work

2.1 Retrieval-Augmented Generation

Lewis et al. (2020) introduced RAG as a general framework combining parametric and non-parametric memory for knowledge-intensive NLP tasks. RAG systems retrieve relevant documents from an indexed corpus and condition generation on the retrieved context, reducing hallucination and enabling knowledge updates without model retraining. Subsequent work has extended RAG to code domains, including repository-level code completion (Zhang et al., 2023), bug fixing (Meng et al., 2024), and developer question answering.

Hybrid retrieval systems combining sparse keyword matching (BM25) with dense vector retrieval via FAISS have shown consistent improvements over either approach alone. Cormack et al. (2009) formalized Reciprocal Rank Fusion (RRF) as a robust rank combination method that is robust across score scales and requires no parameter tuning. Subsequent analysis by Raudaschl et al. (2023) confirmed RRF's effectiveness in hybrid retrieval pipelines.

2.2 Chunking Strategies for RAG

Chunking strategies for text RAG range from fixed-size splitting to semantic and structural methods. Fixed-size chunking with `RecursiveCharacterTextSplitter` is the dominant baseline in production RAG systems, offering simplicity but sacrificing semantic coherence at chunk boundaries. Structure-aware approaches, including paragraph-based, section-based, and hierarchical chunking, have been shown to improve retrieval quality for structured text documents.

For code, the challenge is more acute: syntactic correctness and semantic coherence are inseparable. A chunk that begins in the middle of a function body is syntactically invalid and semantically incomplete. The cAST system (Xie et al., 2025) demonstrated that AST-based chunking, which preserves function and class boundaries, improves retrieval and generation in repository-level tasks. Our work extends this line of

research by decomposing the mechanism of improvement and by conducting a three-phase evaluation that separately assesses retrieval, structural integrity, and generation.

2.3 Evaluation of RAG Systems

Evaluating RAG systems requires metrics across multiple dimensions. Es et al. (2023) introduced RAGAS, a reference-free evaluation framework that assesses Faithfulness (whether generated answers are grounded in retrieved context) and Answer Relevance (whether answers address the query) without requiring human-annotated ground truth. Salemi and Zamani (2024) proposed eRAG, which evaluates retrieval quality by measuring the downstream task performance attributable to each retrieved document. Our evaluation uses retrieval metrics (Hit Rate, MRR, nDCG) alongside RAGAS-inspired generation metrics.

3. Methodology

3.1 Dataset

We evaluate on `psf/requests v2.7.0`, a widely-used Python HTTP library comprising eight source files: `adapters.py`, `api.py`, `auth.py`, `cookies.py`, `hooks.py`, `models.py`, `sessions.py`, and `utils.py`. This library provides a well-structured, real-world codebase with diverse code patterns including class hierarchies, module-level functions, and complex method bodies.

We construct a query set of 32 human-authored queries in English, stratified by difficulty (easy, medium, hard) and annotated with gold symbols — the specific class or function names that contain the answer to each query. Queries span the full range of library functionality, from simple property accessors to complex multi-step processes such as redirect handling and digest authentication.

3.2 Chunking Strategies

Structure-Aware Chunking (SAC) traverses the Python AST using the `ast` module to identify function (`FunctionDef`, `AsyncFunctionDef`) and class (`ClassDef`) boundaries. Each function definition is extracted as a primary chunk unit, with hierarchical parent metadata injected in the form `class:X > function:Y`. Class-level attributes and docstrings are grouped as class header chunks. For AST nodes exceeding the chunk size budget, a fixed-size fallback is applied within the node boundary, preserving the parent metadata on all resulting sub-chunks. Chunks below 50 tokens are merged with adjacent chunks sharing the same parent.

Fixed-Size Chunking (FSC) applies LangChain's `RecursiveCharacterTextSplitter` uniformly across the source files, with no structural awareness or metadata injection.

Both strategies are evaluated under four chunk size budgets: 128, 256, 500, and 1500 tokens.

3.3 Retrieval System

We employ a hybrid retrieval system combining BM25 sparse retrieval with FAISS dense retrieval, fused via Reciprocal Rank Fusion (RRF, $k=60$). Two embedding models are evaluated:

- **MiniLM:** `sentence-transformers/all-MiniLM-L6-v2` (384-dimensional, general-purpose)
- **Jina:** `jinaai/jina-embeddings-v2-base-code` (768-dimensional, code-specialized)

FAISS indices use exact cosine similarity (IndexFlatL2). Top-5 retrieval (k=5) is used for all evaluations.

3.4 Evaluation Metrics

Phase 1 — Retrieval Quality. A retrieved chunk is considered relevant if and only if the gold symbol (class or function name from the query annotation) is present in the chunk’s structural parent metadata. This symbol-level strict matching criterion measures precise structural unit retrieval. We report Hit Rate@1, Hit Rate@5, Mean Reciprocal Rank (MRR), and nDCG@10.

Phase 2 — Structural Integrity. We assess two metrics: Syntax Error Rate (proportion of chunks failing two-step AST parsing: direct parse after dedent, then re-parse within a wrapper function) and Orphan Code Ratio (proportion of chunks with unresolvable structural context, using metadata flags for SAC and indentation heuristics for FSC).

Phase 3 — Generation Quality. We use OpenAI GPT-4o-mini as the generation LLM. Answer Relevance is measured as cosine similarity between query and answer embeddings. Faithfulness is measured as the fraction of answer sentences semantically supported by at least one retrieved chunk (cosine similarity ≥ 0.35). The generation prompt instructs the model to use retrieved context as primary reference with partial-evidence tolerance.

4. SAC Mechanism Analysis: Fallback Chunking

Before presenting results, we characterize a critical implementation property of SAC that affects interpretation of all three evaluation phases.

SAC’s fallback mechanism applies fixed-size splitting within AST nodes that exceed the chunk_size budget. Table 1 shows the proportion of chunks produced via this fallback across chunk size conditions.

Table 1: SAC Fallback Chunking Distribution

Chunk Size	Total Chunks	AST-boundary	Fallback (fixed-size)	Fallback Rate
128	1,369	30	1,339	97.8%
256	716	65	651	90.9%
500	389	117	272	69.9%
1500	216	180	36	16.7%

At chunk_size=128, 97.8% of SAC chunks are produced via fallback splitting. This is expected behavior: the requests v2.7.0 library contains many functions whose bodies exceed 128-character boundaries. Critically, fallback chunks retain the AST parent metadata of their source unit, preserving structural provenance even when boundary-level segmentation is unavailable.

This analysis reveals two independent mechanisms through which SAC achieves retrieval advantage over FSC:

- **Layer 1 — Metadata preservation:** Active at all chunk sizes. Even fallback-split chunks carry `class:X > function:Y` parent metadata, which FSC chunks entirely lack.

- **Layer 2 — AST boundary segmentation:** Active primarily at chunk_size ≥ 500, where 30–83% of chunks are genuine AST-boundary units.

5. Phase 1: Retrieval Quality

5.1 Main Results

Table 2 presents retrieval quality results across all chunk size and embedding model conditions.

Table 2: Retrieval Quality Results (Hit Rate@1, Hit Rate@5, MRR, nDCG@10)

Method	Chunk Size	Embedding	Hit@1	Hit@5	MRR	nDCG@10
SAC	128	MiniLM	0.2500	0.4688	0.3354	0.3610
SAC	128	Jina	0.2500	0.5625	0.3578	0.4085
Fixed	128	MiniLM	0.1250	0.1562	0.1313	0.1371
Fixed	128	Jina	0.0938	0.1875	0.1276	0.1426
SAC	256	MiniLM	0.1875	0.4375	0.2745	0.3062
SAC	256	Jina	0.2812	0.5000	0.3568	0.3876
Fixed	256	MiniLM	0.0938	0.1875	0.1260	0.1374
Fixed	256	Jina	0.0938	0.1562	0.1198	0.1272
SAC	500	MiniLM	0.2188	0.4375	0.3073	0.3323
SAC	500	Jina	0.2812	0.6875	0.4068	0.4817
Fixed	500	MiniLM	0.0938	0.1875	0.1354	0.1483
Fixed	500	Jina	0.1250	0.2500	0.1745	0.1917
SAC	1500	MiniLM	0.2812	0.5312	0.3563	0.3893
SAC	1500	Jina	0.2188	0.6562	0.3786	0.4516
Fixed	1500	MiniLM	0.1875	0.2500	0.2188	0.2209
Fixed	1500	Jina	0.2188	0.2812	0.2370	0.2418

SAC outperforms FSC across all chunk sizes, embedding models, and metrics without exception. The best configuration — SAC + Jina + chunk_size=500 — achieves Hit Rate@5 = 0.6875, MRR = 0.4068, and nDCG@10 = 0.4817.

5.2 SAC vs. FSC Comparison

Table 3 summarizes the SAC advantage in Hit Rate@5 by chunk size and embedding model, with the primary mechanism driving the advantage at each condition.

Table 3: SAC vs. FSC Hit Rate@5 Improvement

Chunk Size	FSC (Jina)	SAC (Jina)	Improvement	Primary Mechanism
128	0.1875	0.5625	+200.0%	Metadata only
256	0.1562	0.5000	+220.2%	Metadata only
500	0.2500	0.6875	+175.0%	Metadata + Boundary
1500	0.2812	0.6562	+133.4%	Metadata + Boundary

At chunk_size=128 and 256, where fallback rates exceed 90%, SAC still achieves 2–3× higher Hit Rate@5. Since segmentation is nearly identical to FSC at these sizes, this advantage is attributable to **metadata preservation alone**. This demonstrates that structural metadata injection is itself a primary driver of retrieval quality, independent of boundary-level segmentation.

5.3 Embedding Model Interaction

The Jina code-specialized embedding model disproportionately benefits SAC compared to FSC. At chunk_size=500, switching from MiniLM to Jina improves SAC's Hit Rate@5 by +0.2500, versus only +0.0625 for FSC. This asymmetry confirms that code-specialized embeddings amplify the structural advantage of SAC: Jina's code-aware representations more effectively bridge natural language queries and function-level code chunks, but this benefit is only realizable when chunks are structurally coherent — a condition SAC satisfies more completely.

5.4 Criterion Validation

Under a relaxed file-level matching criterion, both SAC and FSC achieve Hit Rate@5 = 0.9688 regardless of condition. This confirms that the observed retrieval differences reflect genuine structural retrieval capability rather than file-level proximity effects, and validates the necessity of the strict symbol-level criterion.

6. Phase 2: Structural Integrity

6.1 Main Results

Table 4 presents Syntax Error Rate (SER) and Orphan Code Ratio (OCR) across all chunk size conditions.

Table 4: Structural Integrity Results

Method	Chunk Size	Total Chunks	SER (%)	OCR (%)
SAC	1500	216	9.26%	5.09%
Fixed	1500	98	72.45%	0.00%
SAC	500	389	41.13%	5.66%
Fixed	500	329	64.13%	0.00%
SAC	256	716	57.68%	5.17%
Fixed	256	682	66.13%	0.00%

Method	Chunk Size	Total Chunks	SER (%)	OCR (%)
SAC	128	1369	61.43%	5.26%
Fixed	128	1375	63.56%	0.00%

6.2 Syntax Error Rate Analysis

At chunk_size=1500, SAC achieves a SER of 9.26% versus FSC's 72.45%, a reduction of 63.19 percentage points. This dramatic improvement reflects SAC's alignment of chunk boundaries with complete function-level units at large chunk sizes (83.3% genuine AST-boundary chunks).

As chunk size decreases, SAC's SER rises substantially: from 9.26% at 1500 to 61.43% at 128. This convergence is caused by the fallback splitting mechanism: when function bodies exceed the budget, intra-function splitting produces syntactically incomplete sub-chunks. The SAC advantage over FSC narrows to 2.13 percentage points at chunk_size=128, consistent with the near-complete fallback rate of 97.8% observed at that size.

The SER pattern mirrors the fallback analysis: structural integrity is maximized when chunk size is sufficient to accommodate complete function-level units without triggering fallback.

6.3 Orphan Code Ratio

FSC reports 0.00% OCR across all conditions. This is a detection artifact: the indentation-based heuristic applied to FSC chunks cannot identify orphan code without structural metadata. FSC's high SER values (63–72%) provide indirect evidence of widespread structural fragmentation. SAC's stable OCR of approximately 5% reflects genuine module-level code (import blocks, constants, top-level assignments) that is structurally orphaned by design in the source codebase.

7. Phase 3: Generation Quality

7.1 Main Results

Table 5 presents generation quality results using GPT-4o-mini with Jina-indexed retrieval.

Table 5: Generation Quality Results (Answer Relevance, Faithfulness)

Method	Chunk Size	Answer Relevance	Faithfulness
SAC	128	0.4870	0.5755
SAC	256	0.4755	0.5462
SAC	500	0.6453	0.7389
SAC	1500	0.5984	0.6745
Fixed	128	0.4348	0.5938
Fixed	256	0.5063	0.6102
Fixed	500	0.6816	0.8174

Method	Chunk Size	Answer Relevance	Faithfulness
Fixed	1500	0.6119	0.6732

7.2 Optimal Chunk Size

Both SAC and FSC peak at chunk_size=500 across both metrics. This convergence suggests that 500 tokens provides sufficient contextual content for LLM generation across both chunking strategies. At smaller chunk sizes, both strategies show reduced performance regardless of chunking method. At chunk_size=1500, performance declines from the 500-token peak, likely due to context window length effects reducing the LLM's uniform utilization of all retrieved tokens.

7.3 FSC Generation Advantage

FSC outperforms SAC on Faithfulness at chunk sizes 128, 256, and 500, with the largest gap at chunk_size=500 (Fixed: 0.8174 vs. SAC: 0.7389, Δ=0.0785). At chunk_size=1500, the two strategies converge (SAC: 0.6745, Fixed: 0.6732).

This result appears to contradict Phase 1 findings. The explanation lies in the metric semantics. RAGAS Faithfulness measures whether LLM answer sentences are semantically consistent with retrieved context chunks — not whether the retrieved chunks contain the ground-truth answer. FSC's larger chunks, even when retrieved via lower-precision retrieval, provide denser contextual content per chunk. This allows the LLM to generate plausible, context-consistent answers even when the precise ground-truth unit was not retrieved. SAC's smaller, structurally precise chunks offer more focused but narrower context; when retrieval partially fails, there is less fallback content for the LLM to utilize.

This finding reveals a fundamental trade-off between chunking objectives:

- **For retrieval:** Structural precision and metadata preservation are critical (SAC advantage: 2–3×).
- **For generation:** Per-chunk contextual density is critical (FSC advantage at most chunk sizes).

These objectives are in tension: fine-grained structural chunking improves retrieval precision but reduces contextual density per retrieved unit.

8. Discussion

8.1 The Dual Role of Metadata

Our mechanism decomposition analysis (Section 4) establishes that SAC's retrieval advantage is not monolithic but operates through two independent mechanisms. At small chunk sizes where fallback rates exceed 90%, metadata preservation alone — without any boundary-level segmentation advantage — produces 2–3× higher Hit Rate@5. This is a practically important finding: it implies that even systems constrained to fixed-size chunking can substantially improve retrieval quality by injecting structural provenance metadata into each chunk. The metadata (class:X > function:Y) transforms an anonymous text fragment into a structurally identified unit that can be precisely matched against symbol-level queries.

At larger chunk sizes (500, 1500 tokens), boundary-level segmentation provides a compounding advantage, as structurally complete function units form more coherent embeddings and are more likely to contain

complete answers to queries.

8.2 The Retrieval-Generation Trade-off

Our three-phase evaluation reveals a tension that has not been prominently addressed in prior code RAG literature. SAC's 2–3× retrieval precision advantage does not translate into uniform generation quality improvement. The RAGAS Faithfulness metric, which measures answer-context consistency rather than factual correctness, rewards contextual density over structural precision. This creates a systematic measurement bias in favor of larger chunks, independent of whether those chunks contain the correct information.

This trade-off has practical design implications. Systems optimized purely on generation quality metrics may be led toward larger chunk sizes that sacrifice retrieval precision. A hybrid approach — SAC-based retrieval with context expansion at generation time (retrieving structurally precise chunks and then augmenting with surrounding context) — would preserve retrieval precision while providing sufficient contextual density for generation.

8.3 Consistency with Prior Work

Our retrieval findings are consistent with cAST (Xie et al., 2025), which demonstrated that AST-based chunking improves Recall@5 by 4.3 points on RepoEval. We extend this finding by isolating metadata as an independent retrieval improvement mechanism and by demonstrating that the advantage is robust across four chunk size conditions. Our generation findings partially diverge from cAST, which reported generation improvements; this may reflect differences in the generation task (code completion vs. question answering) and evaluation metric (Pass@1 vs. RAGAS Faithfulness).

9. Limitations

Single codebase. All experiments are conducted on psf/requests v2.7.0. The relatively short functions in this library result in high fallback rates at small chunk sizes. Codebases with longer functions would exhibit different fallback distributions and potentially larger boundary-level effects.

Query set size. The 32-query evaluation set provides directional evidence but may not reach statistical significance for small performance differences.

Single embedding model family. While we compare general-purpose and code-specialized embeddings, the scope does not include graph-based or cross-modal code representations.

Generation metric sensitivity. RAGAS Faithfulness is sensitive to chunk size in ways that may not reflect practical usefulness. Ground-truth answer evaluation would provide stronger evidence of generation quality.

No cross-language evaluation. All experiments use Python source code. Generalizability to other programming languages with different syntactic structures has not been verified.

10. Conclusion

We present a systematic empirical comparison of Structure-Aware Chunking (SAC) and Fixed-Size Chunking (FSC) for code RAG, evaluated across retrieval quality, structural integrity, and generation quality. Our key findings are:

1. **SAC achieves 2.0–3.2× higher Hit Rate@5** than FSC across all evaluated chunk sizes and embedding models, with the best configuration (SAC + Jina + chunk_size=500) achieving Hit Rate@5 = 0.6875.
2. **Structural metadata preservation is an independent retrieval improvement mechanism.** At chunk_size=128, where 97.8% of SAC chunks are produced via fixed-size fallback, SAC still achieves 3× higher Hit Rate@5 than FSC. This demonstrates that metadata injection alone provides substantial retrieval gains independent of boundary-level segmentation.
3. **SAC reduces Syntax Error Rate by up to 63.19 percentage points** at chunk_size=1500, confirming substantial structural integrity advantages at moderate to large chunk sizes.
4. **A fundamental trade-off exists between retrieval precision and generation contextual density.** FSC's larger chunks provide denser context for LLM generation, yielding higher RAGAS Faithfulness at most chunk sizes despite lower retrieval precision. At chunk_size=1500, the two strategies converge.

These findings support a design principle that chunking for code RAG should be treated as a structural decision with two independent levers: metadata injection (beneficial at all chunk sizes) and AST boundary alignment (beneficial when chunk budgets are sufficient). Future work should explore hybrid retrieval-with-expansion architectures that preserve SAC's retrieval precision while addressing the contextual density advantage of larger chunks at generation time.

References

- Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 758–759.
- Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pp. 150–158.
- Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474.
- Robertson, S., and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.
- Salemi, A., and Zamani, H. (2024). Evaluating retrieval quality in retrieval-augmented generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2395–2400.

Schuhmann, B., and Sturua, L., et al. (2023). Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents. arXiv preprint arXiv:2310.19923.

Xie, Y., et al. (2025). cAST: Enhancing code retrieval-augmented generation with structural chunking via abstract syntax tree. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 430–445.

Zhang, F., Chen, B., Zhang, Y., Liang, J., Tan, B., Chen, Y., Huang, F., and Li, Y. (2023). RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2471–2484.